UNIVERSIDAD AUTÓNOMA DE CHIHUAHUA

FACULTAD DE INGENIERÍA

ACTIVIDAD 1: MONGODB CLUSTER

Docente: De Lira Miramontes José Saul

Integrantes:

Amir Ovedi Mendiaz Rodriguez -385685

Santiago de la Mora Martínez -385613

Jesús Manuel Villegas Terrazas -385743

Grupo: 5k2

29/10/26

Reporte MongoDb Clúster

Creación de los nodos necesarios

Para esta actividad se usó un archivo docker-compose el cual nos permitió levantar todos los contenedores necesarios para esta pequeña base de datos distribuida. Dentro de los servicios declarados se encuentra el servidor de configuración, encargado de almacenar los metadatos del clúster, los tres shards, y Mongos que es el enrutador.

Archivo docker-compose:

```
services:
  configsvr:
    image: mongo:7.0
    command: --configsvr --replSet configReplSet --port 27017
    container_name: configsvr
    ports:
      - "27017:27017"
    volumes:
      - configsvr:/data/db

  shard1:
    image: mongo:7.0
    command: --shardsvr --replSet shard1ReplSet --port 27018
    container_name: shard1
    ports:
      - "27018:27018"
    volumes:
      - shard1:/data/db

  shard2:
    image: mongo:7.0
    command: --shardsvr --replSet shard2ReplSet --port 27019
    container_name: shard2
    ports:
      - "27019:27019"
    volumes:
      - shard2:/data/db

  shard3:
    image: mongo:7.0
    command: --shardsvr --replSet shard3ReplSet --port 27020
```

```
    container_name: shard3
    ports:
      - "27020:27020"
    volumes:
      - shard3:/data/db

  mongos:
    image: mongo:7.0
    container_name: mongos
    depends_on:
      - configsvr
      - shard1
      - shard2
      - shard3
    command: >
      mongos --configdb configReplSet/configsvr:27017 --bind_ip_all --
port 27021
    ports:
      - "27021:27021"

volumes:
  configsvr:
  shard1:
  shard2:
  shard3:
```

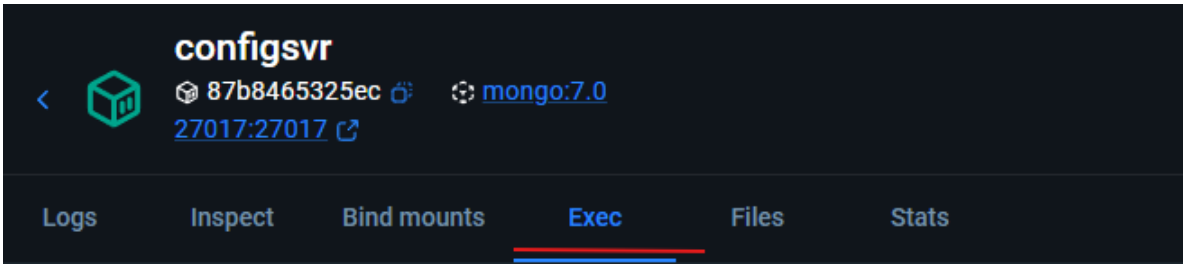
Ejecución

Una vez hecho docker compose sobre nuestro archivo se generaron correctamente todos los servicios. Pudimos corroborar esto fácilmente al entrar a Docker Desktop donde se listan los contenedores.



Config-Server

Dentro de la consola de mongosh de nuestro servidor de configuración ejecutamos este pequeño script de configuración. Donde como se puede observar indicamos con la opción **configsvr: true** que esta instancia de mongo se trata del servidor de configuración.




```
test> rs.initiate({
| _id: "configReplSet",
| configsvr: true,
| members: [{ _id: 0, host: "configsvr:27017" }]
| })
{ ok: 1 }
configReplSet [direct: other] test>
```

Inicializar los shards

Configurar shards

Para configurar el shard1 accedimos a la terminal de mongosh y ejecutamos el siguiente script. En este caso y para los siguientes tuvimos que especificar el puerto, pues ya no está mapeado al 27017 de nuestra computadora. El procedimiento fue totalmente análogo para los shards 2 y 3 .

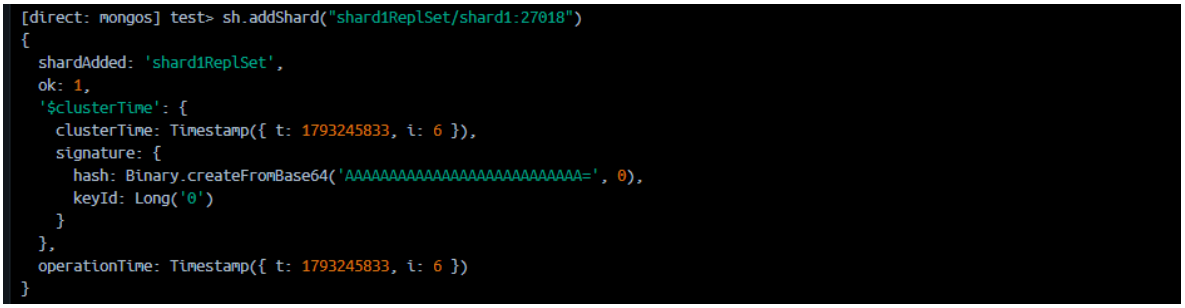
```
# mongosh --port 27018
Current Mongosh Log ID: 6ae2bfddf6025ca1e8f38d6f
Connecting to:          mongodb://127.0.0.1:27018/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.14.0
Using MongoDB:          7.0.45
Using Mongosh:          2.14.0
```

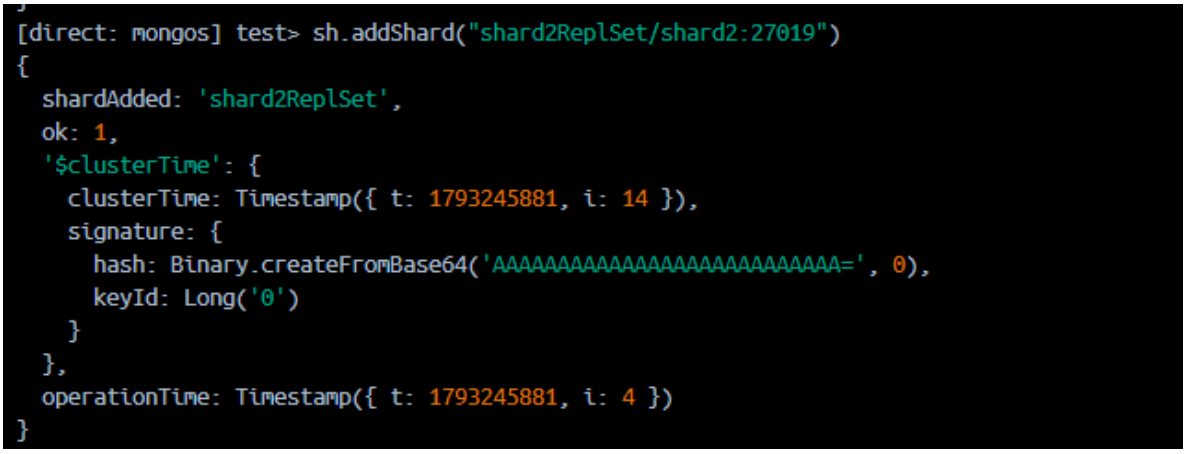
```
test> rs.initiate({
| _id: "shard1ReplSet",
| members: [{ _id: 0, host: "shard1:27018" }]
| })
{ ok: 1 }
shard1ReplSet [direct: other] test>
```

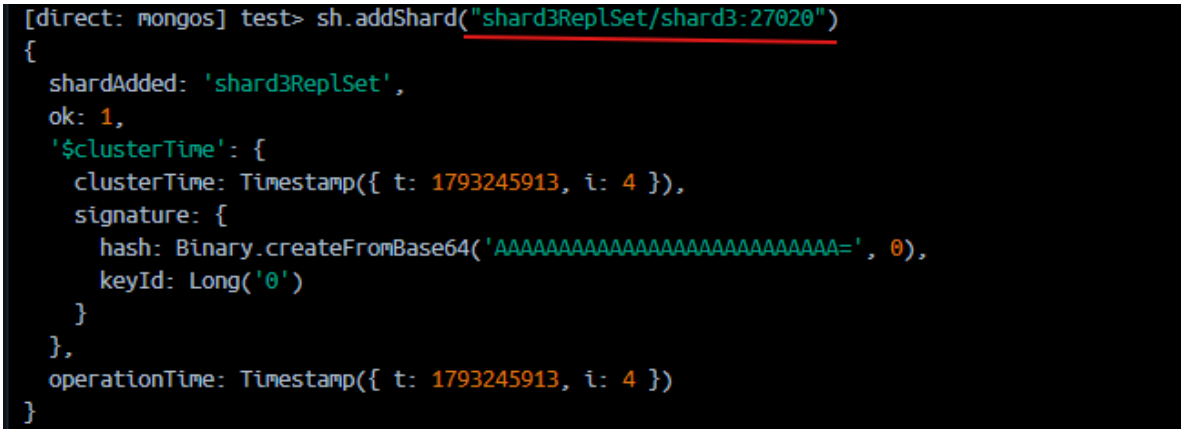
Agregar los shards a mongos

La instancia de mongos quedo en el puerto 27021, así que como vemos en la imagen accedimos a **mongosh** por este puerto para acceder a la terminal. Luego se procedió a agregar cada uno de los shards:



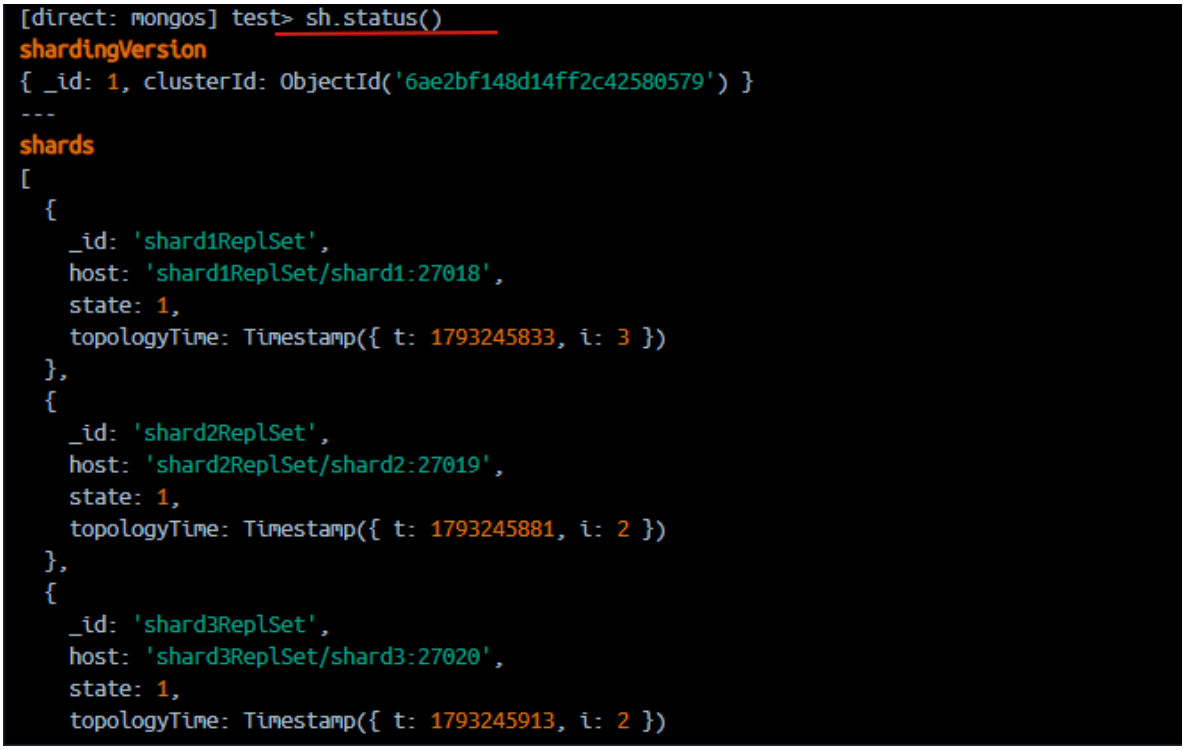






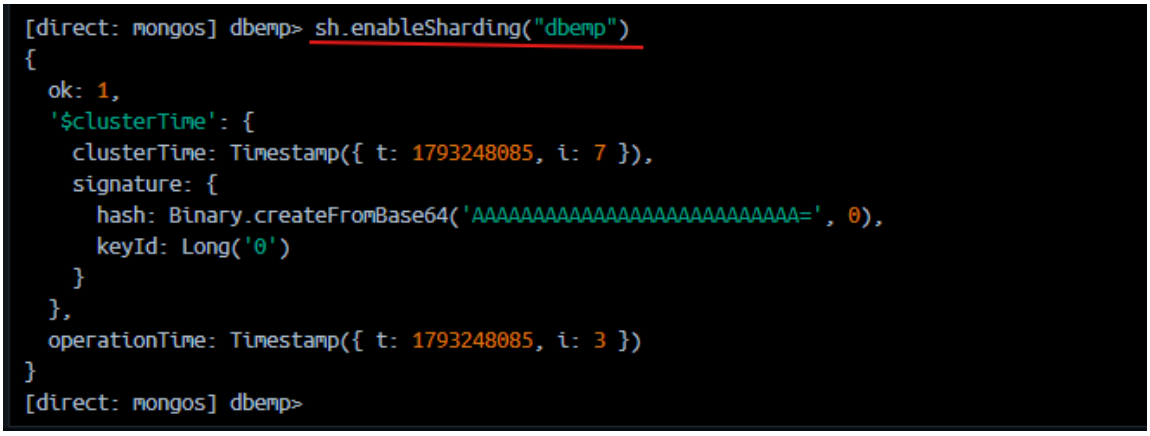
Verificar los shards agregados a mongos

Para verificar que los shards se agregaron con éxito a mongos se ejecutó el comando **sh.status(),** que como se muestra en la imagen nos lista los shards que añadimos en el apartado anterior.



Crear la base de datos

Con el comando **sh.enableSharding()** creamos la base de datos en caso de no existir, como fue nuestro caso, y además este comando es importante porque habilita el particionamiento.



Llave de fragmentación

Creamos la colección empleados **emp** y definimos la llave de fragmentación como hashed, de esta forma los datos se distribuirán de manera uniforme.

```
[direct: mongos] dbemp> db.emp.createIndex({ empno: "hashed" })
| sh.shardCollection("dbemp.emp", { empno: "hashed" })
{
  collectionsharded: 'dbemp.emp',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793248135, i: 50 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793248135, i: 50 })
}
```

Inserción de datos de prueba

Para esta actividad se usó la colección empleados que hemos utilizado desde el inicio del curso. Como podemos ver esto se hace de la forma habitual, es decir haciendo uso de insertMany sobre la colección.

```
[direct: mongos] dbemp> db.emp.insertMany([
|   { empno: 7369, ename: "SMITH", job: "CLERK", sal: 800.0, dept: { deptno: 20, dname: "RESEARCH", loc: "DALLAS" } },
|   { empno: 7499, ename: "ALLEN", job: "SALESMAN", sal: 1600.0, dept: { deptno: 30, dname: "SALES", loc: "CHICAGO" } },
|   { empno: 7521, ename: "WARD", job: "SALESMAN", sal: 1250.0, dept: { deptno: 30, dname: "SALES", loc: "CHICAGO" } },
|   { empno: 7566, ename: "JONES", job: "MANAGER", sal: 2975.0, dept: { deptno: 20, dname: "RESEARCH", loc: "DALLAS" } },
|   { empno: 7654, ename: "MARTIN", job: "SALESMAN", sal: 1250.0, dept: { deptno: 30, dname: "SALES", loc: "CHICAGO" } },
|   { empno: 7698, ename: "BLAKE", job: "MANAGER", sal: 2850.0, dept: { deptno: 30, dname: "SALES", loc: "CHICAGO" } },
|   { empno: 7782, ename: "CLARK", job: "MANAGER", sal: 2450.0, dept: { deptno: 10, dname: "ACCOUNTING", loc: "NEW YORK" } },
|   { empno: 7788, ename: "SCOTT", job: "ANALYST", sal: 3000.0, dept: { deptno: 20, dname: "RESEARCH", loc: "DALLAS" } },
|   { empno: 7839, ename: "KING", job: "PRESIDENT", sal: 5000.0, dept: { deptno: 10, dname: "ACCOUNTING", loc: "NEW YORK" } },
|   { empno: 7844, ename: "TURNER", job: "SALESMAN", sal: 1500.0, dept: { deptno: 30, dname: "SALES", loc: "CHICAGO" } },
|   { empno: 7876, ename: "ADAMS", job: "CLERK", sal: 1100.0, dept: { deptno: 20, dname: "RESEARCH", loc: "DALLAS" } },
|   { empno: 7900, ename: "JAMES", job: "CLERK", sal: 950.0, dept: { deptno: 30, dname: "SALES", loc: "CHICAGO" } },
|   { empno: 7902, ename: "FORD", job: "ANALYST", sal: 3000.0, dept: { deptno: 20, dname: "RESEARCH", loc: "DALLAS" } },
|   { empno: 7934, ename: "MILLER", job: "CLERK", sal: 1300.0, dept: { deptno: 10, dname: "ACCOUNTING", loc: "NEW YORK" } }
| ])
```

Evidencia distribución

Al final pudimos comprobar que efectivamente los documentos se repartieron entre todos los shards al ejecutar el comando getShardDistribution()

```
[direct: mongos] dbemp> db.emp.getShardDistribution()
Shard shard2ReplSet at shard2ReplSet/shard2:27019
{
  data: '670B',
  docs: 5,
  chunks: 2,
  'estimated data per chunk': '335B',
  'estimated docs per chunk': 2
}
---
Shard shard3ReplSet at shard3ReplSet/shard3:27020
{
  data: '545B',
  docs: 4,
  chunks: 2,
  'estimated data per chunk': '272B',
  'estimated docs per chunk': 2
}
---
Shard shard1ReplSet at shard1ReplSet/shard1:27018
```

Resultados

Se observa una distribución uniforme en los shards, esto gracias a que se definió la clave de fragmentación como hashed.

```
Totals
{
  data: '1KiB',
  docs: 14,
  chunks: 6,
  'Shard shard2ReplSet': [
    '35.48 % data',
    '35.71 % docs in cluster',
    '134B avg obj size on shard'
  ],
  'Shard shard3ReplSet': [
    '28.86 % data',
    '28.57 % docs in cluster',
    '136B avg obj size on shard'
  ],
  'Shard shard1ReplSet': [
    '35.64 % data',
    '35.71 % docs in cluster',
    '134B avg obj size on shard'
  ]
}
```